



ITES
RENÉ DESCARTES

CARRERA:
LICENCIATURA EN PROGRAMACIÓN
Y WEB MASTER

DOCENTE:
Mario Alonso Segovia Gutiérrez

MATERIA:
Proyecto de una Plataforma Virtual

ALUMNO:
CASTILLO PINZON CARLOS ALBERTO

FECHA:
03-07-2026

Introducción

En el desarrollo de software es muy común que varias personas trabajen al mismo tiempo sobre un mismo proyecto. Cuando no existe una buena organización para administrar el código, pueden aparecer problemas como pérdida de información, errores o conflictos entre los cambios realizados por cada integrante del equipo.

Para evitar estas situaciones existen los sistemas de control de versiones, como Git, que permiten organizar el trabajo, llevar un historial de modificaciones y facilitar la colaboración entre los desarrolladores. En este documento se analiza el caso del proyecto "SmartLibrary", los problemas que surgieron y una propuesta para mejorar la forma de trabajo utilizando Git.

Parte 1. Análisis del Problema

1. ¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?

El principal error fue que todos trabajaban descargando y reemplazando carpetas completas del proyecto. Esto provocó que algunos cambios se perdieran o fueran reemplazados por otros. Además, no existía una forma de controlar quién modificó cada archivo ni cuándo se hicieron esos cambios.

2. ¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?

Porque al momento del desarrollo del trabajo se trabaja por medio de copias y esto puede hacer que al momento de subir a la carpeta se eliminen cambios o simplemente no se hagan algunos cambios sin que las personas lo detecten sin poder ver historial de los cambios haciendo conflictos al momento de unir partes del proyecto.

3. ¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?

En proyectos grandes los problemas serían mucho mayores. Se podrían perder semanas de trabajo, aumentarían los errores del sistema y sería muy difícil encontrar la causa de un problema. También habría retrasos en las entregas y el trabajo del equipo sería menos organizado y eficiente.

Parte 2. Importancia del Control de Versiones

- **¿Qué es un sistema de control de versiones?**

La principal función es que permite administrar y poder guardar los cambios de un proyecto y ayuda a que varias personas puedan trabajar al mismo tiempo sin afectar el trabajo.

- **¿Cuál es su principal objetivo?**

Su principal objetivo es organizar el desarrollo del proyecto, mantener un historial de cambios y facilitar el trabajo colaborativo, evitando la pérdida de información.

- **¿Qué ventajas ofrece frente al manejo tradicional de archivos?**

1. Guarda el historial de todas las modificaciones.

Permite ver todos los cambios que se han hecho en el proyecto y regresar a una versión anterior si es necesario

2. Permite saber quién realizó cada cambio.

Cada modificación queda registrada con el nombre del que la realizó, por lo que es fácil identificar quién hizo cada cambio.

3. Evita que se pierda información.

Los cambios se guardan correctamente, por lo que es menos probable que se elimine o sobrescriba el trabajo de otra persona.

4. Facilita trabajar en equipo.

Cada integrante puede trabajar en su parte del proyecto y después unir sus cambios de forma más organizada y segura.

Parte 3. Propuesta de Organización

- ¿Qué rama utilizarías para la versión estable del sistema?

Utilizaría la rama main, ya que debe contener únicamente versiones estables y completamente probadas del sistema.

- ¿Qué rama utilizarías para integrar el trabajo del equipo?

Utilizaría una rama llamada develop, donde se integrarían las nuevas funciones antes de pasar a la versión estable, esta puede tener variaciones de nombre como pruebas etc...

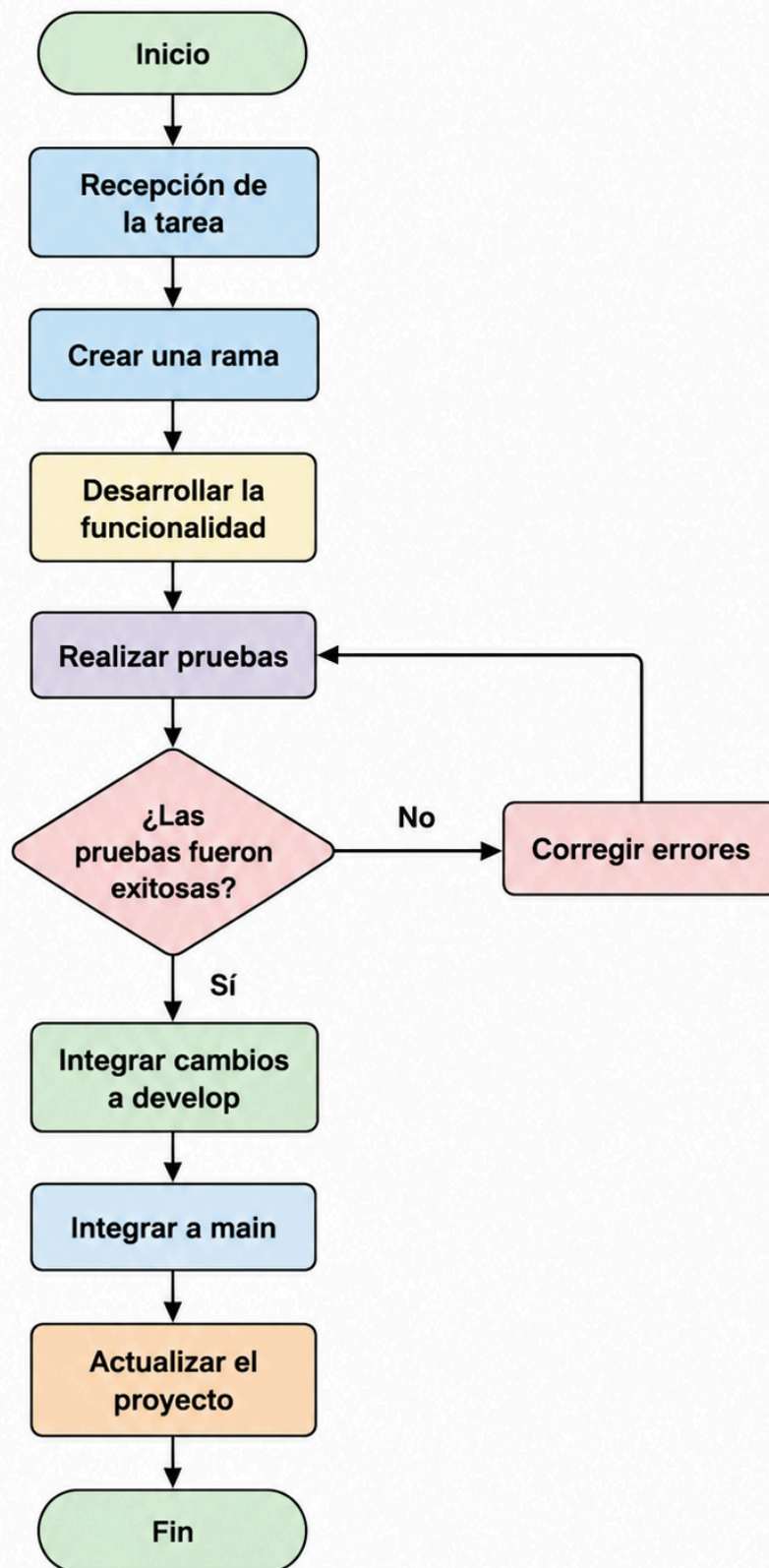
- ¿Cada desarrollador debería trabajar directamente sobre la rama principal?
Explica tu respuesta.

No. Cada desarrollador debería trabajar en una rama independiente para desarrollar su funcionalidad. De esta manera, si ocurre algún error solo afecta su propia rama y no el proyecto principal. Una vez se terminan las pruebas, los cambios pueden integrarse en pruebas.

- ¿Cuándo deberían integrarse los cambios al proyecto?

Los cambios deberían integrarse únicamente cuando la funcionalidad esté terminada, haya sido revisada y pase las pruebas correspondientes.

Parte 4. Flujo de Trabajo



Parte 5. Análisis de situaciones

1. ¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?

Se produciría un conflicto de cambios. Antes de unir el trabajo, ambos desarrolladores tendrían que revisar el archivo y decidir cuáles modificaciones conservar para evitar errores.

2. ¿Por qué es importante conservar un historial de cambios en un proyecto?

Porque permite conocer quién realizó cada modificación, cuándo se hizo y qué cambios se aplicaron. Además, facilita regresar a una versión anterior si aparece algún problema.

3. ¿Qué beneficios aporta trabajar mediante ramas independientes?

Permite que cada desarrollador trabaje sin afectar el trabajo de los demás. También facilita probar nuevas funciones y corregir errores antes de incorporarlas al proyecto principal.

4. ¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?

Permite identificar rápidamente qué cambio produjo el error y regresar a una versión anterior mientras se corrige el problema, evitando afectar el funcionamiento del sistema.

5. ¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?

Porque ayuda a organizar el trabajo del equipo, evita pérdidas de información, facilita la colaboración entre los desarrolladores y mantiene un historial completo de todas las modificaciones realizadas durante el desarrollo.

Conclusión

Con esta actividad aprendí que el control de versiones es una herramienta muy importante para desarrollar software de manera organizada. También comprendí que trabajar únicamente compartiendo carpetas puede ocasionar muchos problemas, especialmente cuando participan varias personas en un mismo proyecto.

Git se ha convertido en un estándar de la industria porque permite administrar los cambios de forma segura, trabajar mediante ramas independientes y mantener un historial completo del proyecto. Gracias a estas funciones, los equipos pueden colaborar de manera más eficiente y reducir los errores durante el desarrollo.

Finalmente, considero que una buena organización del código influye directamente en el éxito de un proyecto, ya que facilita el trabajo en equipo, mejora el mantenimiento del sistema y permite entregar un software más estable y de mejor calidad.